



SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL
SCIENCES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



COURSE NAME: Theory of Computation

COURSE CODE: CSA1308

COURSE OUTCOME COVERED

CO1 -- ASSESSMENT TOOL 2

Apply mathematical foundations such as formal proofs, induction, and basic concepts of automata theory to solve computational problems. (BL: 3)

Assessment Tool Weightage: 30%

QUESTIONS: 5

Total Marks: 50

Duration: 1 Hour

SIMULATION

Code of Conduct:

I, VIRINDA SINGH V (192371012), (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Virinda Singh V.

Reg. No: 192371012.

RUBRICS FOR CALCULATION

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Q4 (10)	Q5 (10)	Total Marks (50)
Conceptual Understanding	30						
Accuracy of Answer	25						
Application of Knowledge	20						
Completeness of Response	15						
Clarity & Presentation	10						
Total per Question	10 Marks	/10	/10	/10	/10	/10	/ 50

1 Design a Non-Deterministic Finite Automaton (NBA) for an Automatic Door System based on sensor inputs. An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states: Closed, Opening, Open, and Closing. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the Closed state. When a person is detected (P), the system may non-deterministically transition from Closed to Opening. Once opened, the system moves to the Open state. If no person is detected (N), the door transitions to Closing and eventually returns to the Closed state. Non-determinism occurs when the system may remain in the Open state even if a person is detected or may start closing if no detection occurs.

2 Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Red). Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

3 Design DFA using simulator to accept the string the end with ab over set {a,b} W= aaabab

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings "c", "bc", and "bcaaa" over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string "c" should be accepted directly from the start state, while "bc" should pass through states representing "b" and then "c". Similarly, "bcaaa" should be processed through a sequence of states that track each additional "a" after "bc". Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

5 Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either "a" or "b". The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either "a" or "b". After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of "a" and "b", thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

1. NFA for an Automatic Door System

Conceptual understanding.

A Non-deterministic Finite Automaton (NFA) is ideal for modeling systems where inputs might not immediately or strictly dictate a single subsequent state due to sensor delays or physical mechanics. In this automatic door system, non-determinism represents the system's "choice" to remain in a state momentarily before transitioning, mimicking real-world sensor polling.

Formal Definition:

Let the NFA be defined as $M = (Q, \Sigma, \delta, q_0, F)$

- States (Q): $\{q_{\text{closed}}, q_{\text{opening}}, q_{\text{open}}, q_{\text{closing}}\}$
- Alphabet (Σ): $\{P, N\}$ where P = Person detected, N = No person detected.
- Start State (q_0) = q_{closed} .
- Accepting States (F): $\{q_{\text{closed}}\}$ (Assuming the system's "rest" or safe baseline state is the accepted completion of a cycle).

Transition Function(δ) & Logic:

1. closed (q_{closed}): On P , it can non-deterministically stay closed for a polling cycle or begin opening.

$$\delta(q_{\text{closed}}, P) = \{q_{\text{closed}}, q_{\text{opening}}\}$$

Q2. Finite Automaton for a Traffic Light Control System.

Conceptual understanding:

A traffic light is best modeled as a finite state machine where transitions are governed by timers (internal signals) and external interruptions (sensor signals). To ensure safe traffic flow, strict order must be maintained, with specific interrupt handlers for edge cases.

Base Automaton Logic:

- States (Q): $\{ \text{Red}, \text{Green}, \text{Yellow} \}$.
- Inputs (Σ): $\{ t_r, t_g, t_y \}$ (timers expiring for Red, Green, Yellow)
- Sequence : $\text{Red} \xrightarrow{t_r} \text{Green} \xrightarrow{t_g} \text{Yellow} \xrightarrow{t_y} \text{Red}$.

Handling Extensions (Application of knowledge)

To Handling continuous cycling alongside pedestrian crossings (P_{press}) and Emergency Vehicles (E_{alert}), We introduce overriding input conditions:

1. **Timing Constraints**: The automation remains in its current state as long as its specific timer $t < T_{\text{max}}$. The transition only occurs when $t = T_{\text{max}}$.

2. **Pedestrian Crossing (P_{press})**: If a Pedestrian presses the button while the light is Green, the automation truncates the Green timer (t_g) and safety forces a transition to Yellow to Prepare for a safe stop

- Transition : $\text{Green} \xrightarrow{P_{\text{press}}} \text{Yellow}$.

3. **Emergency Vehicle Override (E_{alert})**: Safety Requires Immediate action. If an emergency vehicle approaches,

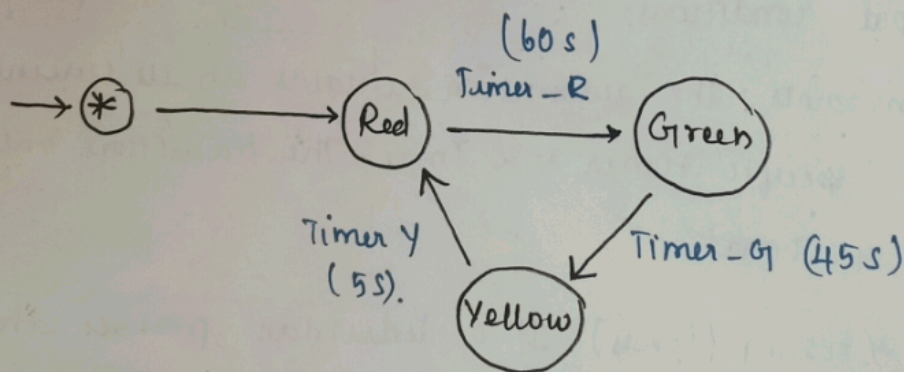
all lights opposing it must immediately cycle to Red.
If our automaton represents the light in the emergency vehicle's path, it must force a Green state.

Transitions: Red $\xrightarrow{\text{Ealert}}$ Green, Yellow $\xrightarrow{\text{Ealert}}$ Green.
Once the emergency clears (Eclear), it resumes normal timer operations.

State Transition Logic.

current state	Timer Expires	Pedestrian (Ppress)	Emergency (Ealert)
Red	$\rightarrow$ Green	No change (wait for Green)	$\rightarrow$ Green.
Green.	$\rightarrow$ Yellow	$\rightarrow$ Yellow (Early)	No change (Stays Green)
Yellow	$\rightarrow$ Red	No change (unsafe to cross)	$\rightarrow$ Green

State transition Diagram



3). DFA to Accept Strings Ending with "ab" over (a,b)

Automation Definition:

$\rightarrow$ State (Q): $\{q_0, q_1, q_2\}$

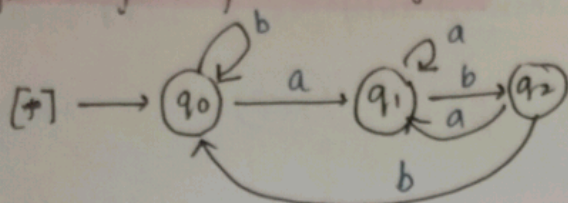
$\rightarrow$ Alphabet (Σ): $\{a, b\}$

$\rightarrow$ Start state (q_0): q_0

→ Accept state (f): $\{q_2\}$

→ Example Trace for $W = aaabab$: $q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2$
 (Accepted)

Design Diagram / State Diagram



Transition table (for simulator Input):

State	Input 'a'	Input 'b'
→ q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

Simulator verification Trace for $W = aaabab$:

- ① Start at q_0 .
- ② Read 'a': $\delta(q_0, a) \rightarrow q_1$
- ③ Read 'a': $\delta(q_1, a) \rightarrow q_1$
- ④ Read 'a': $\delta(q_1, a) \rightarrow q_1$
- ⑤ Read 'b': $\delta(q_1, b) \rightarrow q_2$
- ⑥ Read 'a': $\delta(q_2, a) \rightarrow q_1$
- ⑦ Read 'b': $\delta(q_1, b) \rightarrow q_2$.

Conceptual understanding

The DFA $\rightarrow = 1$ transition / Input for every state. To accept strings ending in "ab", the DFA must "remember" the last one or two characters processed to recognize the sequence at the very end of the string.

Result the Machine ends in state q_2 . Since $q_2 \in F$

$W = aaabab$ is Accepted.

Q4. DFA to accept ONLY "c", "bc", and "bcaaa"

Conceptual understanding:

Because this DFA accepts a finite language (exactly three specific strings). We must construct distinct, linear paths from the start state to accepting states. Any deviation from these exact sequences must route immediately to a "Dead state" (q_d) from which it can never escape.

Formal Definition:

- States (Q): $\{q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_d\}$
- Alphabet (Σ): $\{a, b, c\}$
- Start state: q_0
- Accepting states (F): $\{q_1, q_3, q_6\}$

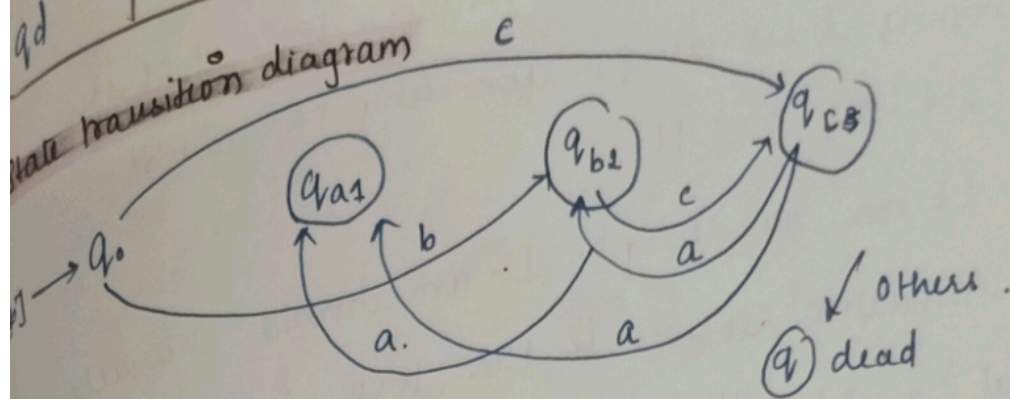
Transition Functions:

- Path for 'c': $q_0 \xrightarrow{c} q_1$ (Accept)
- path for "bc": $q_0 \xrightarrow{b} q_2 \xrightarrow{c} q_3$ (Accept)
- Path for "bcaaa": $q_3 \xrightarrow{a} q_4 \xrightarrow{a} q_5 \xrightarrow{a} q_6$ (Accept)

Transition Table

State	I/p a	I/p b	I/p c
→ q_0	q_d	q_2	q_1
q_1	q_d	q_d	q_d
q_2	q_d	q_d	q_3
q_3	q_4	q_d	q_d
q_4	q_5	q_d	q_d

		q_d	q_d
	q_b	q_d	q_d
	q_d	q_d	q_d
q_c		q_d	q_d
q_d	q_d		



Q5: Conceptual understanding

This DFA requires immediate verification of the first character since the required alphabet is $\{a, b\}$, any valid input string of length ≥ 1 will begin with "a" or "b". The machine transitions to an accepting state upon reading the first symbol and remains there infinitely. A dead state (q_d) handles the empty string (by default of q_0 not being acceptance) or any character outside the valid alphabet input by a simulator.

Formal Definition:

- State (Q): $\{q_0, q_1, q_d\}$
- Alphabet (Σ): $\{a, b, \text{others}\}$
- Start State: q_0
- Accepting State (F): $\{q_1\}$

Transition Rules:

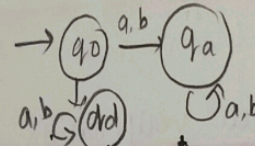
1. Initial Validation: From the start state q_0 , if the input is 'a' or 'b', transition to the accepting state q_1 . $\delta(q_0, a) = q_1$, $\delta(q_0, b) = q_1$

2. Infinite Acceptance: Once in q_1 , ignore the rest of the sequence by looping back to q_1 for any 'a' or 'b'.

$$\delta(q_1, a) = q_1, \delta(q_1, b) = q_1.$$

3. Dead state (Error handling): If an invalid character is fed into the simulator, it goes to the trap state.

$\delta(q_0, \text{invalid}) = q_d$. Once in the dead state, it stays there.

$\delta(q_d, \text{any}) = q_d$.  → state diagram.

Transition Table (for simulator Input):

State	Input 'a'	Input 'b'	Invalid / Empty
→ q_0	q_1	q_1	q_d
∞q_1	q_1	q_1	q_d
q_d	q_d	q_d	q_d

simulator Verification

• Empty string (ϵ): Stays in q_0 . Since $q_0 \notin F$, it is correctly Rejected.

• Valid String ("baab"): $q_0 \xrightarrow{b} q_1 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_1$
Ends in $q_1 \in F$, correctly Accepted.

